
Software Requirements Specification

for

GameGuessr

Version 1.0 approved

Prepared by Group 4

Binus University

25 February 2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	1
1.5 References	1
2. Overall Description	2
2.1 Product Perspective	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment	2
2.5 Design and Implementation Constraints	2
2.6 User Documentation	2
2.7 Assumptions and Dependencies	3
3. External Interface Requirements	3
3.1 User Interfaces	3
3.2 Hardware Interfaces	3
3.3 Software Interfaces	3
3.4 Communications Interfaces	3
4. System Features	4
4.1 System Feature 1	4
4.2 System Feature 2 (and so on)	4
5. Other Nonfunctional Requirements	4
5.1 Performance Requirements	4
5.2 Safety Requirements	5
5.3 Security Requirements	5
5.4 Software Quality Attributes	5
5.5 Business Rules	5
6. Other Requirements	5
Appendix A: Glossary	5
Appendix B: Analysis Models	5
Appendix C: To Be Determined List	6

Revision History

Name	Date	Reason For Changes	Version
Group 4 (Fairy, Jennifer, Jiovanny)	28 February 2026	Writing the introduction	1.1

Group 3 (Albert, Gregorio, Jovan)	31 March 2026	Drafting sections 2.1 and 2.2 to establish the Overall Description.	1.2
Group 3 (Albert, Gregorio, Jovan)	1 April 2026	Rewriting Chapter 2 to provide comprehensive content.	1.3
Group 3 (Albert, Gregorio, Jovan)	8 April 2026	deleting inaccurate information in section 2.1 to ensure correctness.	1.4
Group 3 (Albert, Gregorio, Jovan)	17 April 2026	Replacing Chapter 2 with a new version to add missing details.	1.5
Group 3 (Albert, Gregorio, Jovan)	22 April 2026	Writing Chapters 3 to 5 and adding UML diagrams to finalize system	1.6
Group 3 (Albert, Gregorio, Jovan)	23 April 2026	Polishing the document layout and formatting for consistency.	1.7

1. Introduction

1.1 Purpose

The primary objective of this document is to provide a detailed explanation about GameGuessr. GameGuessr is a website-based quiz game. This document outlines the purpose, features, system interfaces, functionalities and limitations of GameGuessr. It is intended to serve as a crucial roadmap for developers and stakeholders, ensuring effective collaboration and communication throughout the development process. By providing a clear scope about the projects' objectives, this document aims to facilitate seamless development, testing, integration and deployment of the website. In addition, this document serves as a reference for future modifications and improvements, as the system will adapt continuously according to the user feedback. It will be submitted to Binus for its approval.

The release version of GameGuessr is 1.0.0-MVP. The development cycle consists of 4 phases and each phase will be conducted within 1 month. The SRS scope of this product includes 3 main subsystems which are the uploader module, quiz module, and leaderboard module. The development process will be done using an incremental process model, allowing iterative improvement and adaptation at each phase.

1.2 Document Conventions

This Software Requirements Specification (SRS) follows the IEEE 830-1998 standard structure for software requirements documentation. The following conventions used throughout this document are as follows:

Typographical Conventions

This section outlines specific visual styles which are used to highlight different types of technical information.

Element	Convention	Example
Requirement ID	Bold alphanumeric code	REQ-QUIZ-1
Software Modules	Title Case & Bold	The Uploader Module shall...
User Interface (UI)	Open Sans Bold	Click the Submit Quiz button
Data / Variables	Monospaced font (Google sans code)	user_score, leaderboard_rank
Priority Level	Bold within brackets	[High] , [Medium] , [Low]
Emphasis	Italicized text	This action is <i>irreversible</i>

Requirement Priority and Inheritance

The following rules explain how the requirements in GameGuessr are prioritized by using an incremental process model.

- Priority Inheritance: High level requirements set the baseline priority. All detailed sub-requirements within that module *inherit* the high-level priority unless specifically overridden.
- Unique Tagging: Every individual requirement statement must have its own unique ID (e.g., **REQ-QUIZ-1**) to facilitate testing and integration across all 4 phases.

Requirement Statements

This section follows the standards of IEEE 830-1998 terminology to define the necessity of every feature.

- Shall: A mandatory requirement for the 1.0.0-MVP release (High level requirements)
- Should: A recommended feature
- May: An optional feature that may be added based on user feedback.

1.3 Intended Audience and Reading Suggestions

This Software Requirements Specification (SRS) is intended for all parties involved in the design, development, testing, deployment, and evaluation of GameGuessr. The document is organized to support a wide range of readers with different levels of technical expertise. The following outlines the recommended reading approach for each reader type:

- **Developers and Software Engineers**
They should read this document in its entirety, with extra focus on Section 3 (External Interface Requirements) and Section 4 (System Features) for implementation specifications, and Section 5 (Other Nonfunctional Requirements) for quality and performance standards.
- **Project Managers**
To fully understand the project's objectives and Software Development Life Cycle (SDLC), it is recommended to focus on Section 1 (Introduction) and Section 2 (Overall Description) to understand the project scope, constraints, and assumptions relevant to planning and risk management.
- **Quality Assurance (QA) Testers**
They should prioritize Section 4 (System Features) for test case development, supported by Section 3 and Section 5 for interface and non-functional validation. Appendix C (To Be Determined List) should be monitored for deferred items affecting test coverage.
- **Documentation Writers**
They should treat this document as a baseline reference for GameGuessr's original scope. They should read Sections 1 through 2 for the foundational concept, while actively referring to Appendices A through C.
- **Stakeholders and Non-Technical Readers**
For readers with a limited technical background, such as marketing staff and end users, it is suggested that they focus on Section 1 (Introduction) and Section 2 (Overall Description) to gain a sufficient understanding of the software's purpose and scope.

The remainder of this document is organized as follows. Section 2 (Overall Description), of this document gives an overview of the functionality of the product. It describes the informal requirements and is used to establish a context for the technical requirement specifications in the next chapter. Section 3 (External Interface Requirement) of this document details the logical and physical characteristics of the system's user, hardware, software, and communications interfaces of GameGuessr. Section 4 (System Features) is the most technical as it describes the precise functional requirements of GameGuessr's three core subsystems. Section 5 (Other NonFunctional Requirements) explains performance benchmarks, security constraints, safety considerations, and software quality attributes. Section 6 (Other Requirements) captures any remaining legal, regulatory, or domain-specific requirements. The document concludes with 3

appendices: Appendix A (Glossary), Appendix B (Analysis Models), and Appendix C (To Be Determined List).

1.4 Product Scope

GameGuessr is a web-based interactive quiz game platform designed to make learning and knowledge testing more engaging, competitive, and accessible. The platform enables users to participate in quiz sessions generated from uploaded content, competing against others in real time and tracking performance through a dynamic leaderboard system.

The core objective is to bridge the gap between content sharing and interactive assessment by allowing uploaders to convert their materials into quiz questions, which other users can then answer within a timed, game-like environment. This approach transforms passive content consumption into an active, competitive experience that reinforces knowledge retention.

GameGuessr is structured around three primary subsystems:

- **Uploader Module**
→ Allows designated users to submit content and configure quiz parameters.
- **Quiz Module**
→ Delivers questions to participants, manages session timing, records responses.
- **Leaderboard Module**
→ aggregates scores and ranks participants, providing a transparent and motivating competitive layer.

From a strategic standpoint, GameGuessr addresses the growing demand for gamified educational and entertainment tools in digital environments. Its incremental development model ensures that the platform can adapt based on user feedback across its four development phases, with the initial release (v1.0.0-MVP) focusing on delivering a stable, functional core experience.

The platform is intended for two primary user groups: content uploaders who wish to share and test knowledge, and quiz participants seeking an engaging, competitive experience. GameGuessr does not aim to replace formal learning management systems but instead serves as a lightweight, gamified complement to existing educational or entertainment ecosystems.

1.5 References

[1] *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998. [Online]. Available: <http://www.math.uaa.alaska.edu/~afkjm/cs401/IEEE830.pdf> (Accessed: Feb. 25, 2026).

[2] V. Rawal, "1. Introduction 1.1–1.4," Medium, Oct. 6, 2020. [Online]. Available: <https://visheshrawal.medium.com/1-introduction-1-1-41c2e07b6e97> (Accessed: Feb. 25, 2026).

[3] NewLine Technologies, "Incremental Model of Software Development Life Cycle," Medium, Jan. 3, 2018. [Online]. Available: <https://medium.com/@newlinetechnologies/incremental-model-of-software-development-life-cycle-b17a8cffc045> (Accessed: Feb. 26, 2026).

2. Overall Description

2.1 Product Perspective

GameGuessr is a new, self-contained, web-based application. It is not a follow-on to any existing product nor a replacement for a legacy system. It is designed to operate independently, with no mandatory integration with external platforms except for optional email notifications.

System Architecture Context

GameGuessr follows a three-tier client-server architecture:

- **Presentation Tier (Frontend):** Single-Page Application (SPA) built with React.js 18.2.0. It runs entirely in the user's browser and communicates with the backend exclusively via HTTPS.
- **Application Tier (Backend):** Node.js 20 LTS with Express.js 4.18.2 providing RESTful APIs. Real-time communication is handled by a separate WebSocket server using Socket.io 4.7.2.
- **Data Tier:** MySQL 8.0.35 relational database for persistent storage. File uploads (content files) are stored on the server's local filesystem during MVP; a cloud storage migration path is noted in the TBD list.

Block Diagram – Figure 2.1: System Architecture

Fig 2.1 System Architecture Diagram | See Appendix B

- **Tech Stack Details:**

Component	Technology & Version	Justification
Frontend Framework	React 18.2.0	Component reusability, large ecosystem, virtual DOM for performance.
UI Component Library	Material-UI (MUI) v5.14	Provides accessible, pre-built components that match modern design standards.
State Management	React Context API + useReducer	Sufficient for MVP complexity; avoids Redux boilerplate.
Backend Runtime	Node.js 20.11.0 LTS	Non-blocking I/O ideal for real-time apps; large package ecosystem.
API Framework	Express 4.18.2	Minimalist, fast, and widely supported.
Real-time Library	Socket.io 4.7.2	Automatic fallback to long-polling; built-in room support for quiz sessions.
Database	MySQL 8.0.35	ACID compliance, strong community, and familiarity within the development team.

2.2 Product Functions

The following is a high-level summary of the major functions GameGuessr must perform. Detailed specifications appear in Section 4.

- **FUN-01: User Account Management**

Users can create an account, log in, log out, and update basic profile information (display name, avatar URL). The system supports three distinct roles: Participant, Uploader, and Administrator.

- **FUN-02: Content Upload & Quiz Creation**

Uploaders can submit a document file (TXT, PDF, DOCX) or a structured JSON file containing questions. The system parses the content and automatically generates a set of multiple-choice questions. Uploaders can edit, add, or delete auto-generated questions, set a time limit per question (15-120 seconds), and publish the quiz.

- **FUN-03: Real-Time Quiz Participation**

Participants can browse a list of available quizzes, join a quiz session, and answer questions one by one against a synchronized timer. The system records answers instantly via WebSocket, calculates a running score, and shows immediate feedback after each question.

- **FUN-04: Leaderboard and Performance Tracking**

Upon quiz completion, the participant's score is posted to a global leaderboard for that quiz. Participants can view leaderboards, their own quiz history, and summary statistics (e.g., average score, total quizzes taken).

- **FUN-05: Administrative Moderation**

Administrators can view all users, promote/demote user roles, hide or delete inappropriate quizzes, and view system-wide usage statistics.

- **Top-Level Data Flow Diagram (DFD) Context Level**

[Insert a context diagram showing GameGuessr as a single process interacting with Participant, Uploader, and Administrator external entities. Place in Appendix B.]

2.3 User Classes and Characteristics

GameGuessr identifies four primary user classes. Each class differs in terms of access privileges, frequency of use, and technical expertise required.

The system distinguishes four user classes based on privilege level and intended use. The following table provides detailed characteristics.

User Class	Privilege Level	Frequency of Use	Technical Expertise	Key Responsibilities / Use Cases	Importance to MVP

Guest	None	Occasional	Basic web browsing	View landing page, read FAQ, register for an account.	Low
Participant	Low	Regular (daily)	Basic - can click and type	Browse quizzes, join live sessions, answer questions, view personal history and leaderboards.	Critical
Uploader	Medium	Weekly / Monthly	Intermediate - comfortable with file uploads, basic form configuration	Upload content files, create and edit quizzes, monitor performance of their own quizzes.	High
Administrator	Highest	As needed	Advanced - understands system operations, data management	Manage user accounts and roles, moderate content, access system logs and usage reports.	Medium (Support)

Special Notes:

- A **Participant** can be promoted to **Uploader** only by an Administrator.
- All registered users are Participants by default.
- The Administrator account is pre-seeded during deployment; no self-registration to Admin role is possible.

2.4 Operating Environment

2.4.1 Client-Side Environment

Requirement	Specification
Browsers Supported	Google Chrome ≥110, Mozilla Firefox ≥110, Microsoft Edge ≥110, Apple Safari ≥16.0
Operating Systems	Windows 10/11, macOS 12+, Ubuntu 22.04+, Android 12+, iOS 16+

Screen Resolution	Minimum: 360×640 px (mobile-first). Optimal: 1366×768 px and above.
JavaScript	Enabled (ES2020 features required)
Cookies & LocalStorage	Enabled for JWT token persistence and session recovery.
Network	Broadband or 4G connection with at least 1 Mbps down/up. Latency <100 ms to server recommended.
WebSocket Support	Required for real-time quiz features; Socket.io falls back to HTTP long-polling if blocked.

2.4.2 Server-Side Environment

Component	Specification
OS	Ubuntu 22.04.3 LTS (64-bit)
Runtime	Node.js 20.11.0 LTS
Process Manager	PM2 5.3.0 (for production clustering and auto-restart)
Web Server	Nginx 1.24.0 as reverse proxy (terminates HTTPS, serves static assets, proxies to Node.js)
Database	MySQL Community Server 8.0.35
Memory	Minimum 2 GB RAM; recommended 4 GB for production.
Storage	Minimum 20 GB SSD (for OS, application, database, and uploaded files).
Network Ports	443 (HTTPS), 80 (HTTP redirect to HTTPS)

2.4.3 Development Environment

- **Version Control:** Git 2.40+ with remote repository on GitHub/GitLab.
- **IDE:** Visual Studio Code with ESLint and Prettier extensions.
- **Database GUI:** MySQL Workbench 8.0 or DBeaver.
- **API Testing:** Postman or Insomnia.
- **Package Manager:** npm 10.2+

2.5 Design and Implementation Constraints

The following constraints must be observed throughout development and deployment.

- **C-01: Browser-Only Client**

No native iOS or Android application will be developed for the MVP. All user interaction occurs via a web browser. This limits offline capability; a user must be online to participate in quizzes.

- **C-02: WebSocket Dependency**

Real-time question delivery, timer sync, and answer submission require a persistent WebSocket connection. In environments where WebSocket is blocked (e.g., some corporate firewalls), Socket.io will fall back to HTTP long-polling. Performance may degrade in fallback mode, and timers may drift slightly.

- **C-03: File Upload Size Limit**

Uploaded content files shall not exceed 10 MB. This constraint is due to server memory and processing limitations during text extraction and question generation.

- **C-04: Single Language Interface**

The user interface, system messages, and error notifications are presented in English only. Content (quiz questions, answers) may be in any language (including Indonesian), but the UI is fixed to English for MVP.

- **C-05: Incremental Development Phases**

The project is delivered in four monthly increments. The SRS describes the MVP (Phase 1) features only. Features planned for Phases 2-4 (e.g., social sharing, advanced analytics, custom themes) are out of scope for this document.

- **C-06: Single-Server Architecture**

The entire application stack (Node.js API, WebSocket server, MySQL) runs on a single virtual machine instance. This simplifies deployment and cost but limits horizontal scalability. Future phases may migrate to separate database and caching servers.

- **C-07: No Third-Party Authentication**

The MVP will not support OAuth (Google, Facebook) login. Only email/password registration is implemented. This reduces integration complexity.

- **C-08: File Storage – Local Filesystem**

Uploaded files are stored on the server's local disk (/var/www/uploads). A cloud storage solution (e.g., AWS S3) is deferred to a later phase.

2.6 User Documentation

The following deliverables will be provided to end-users and system administrators.

Document / Component	Format	Audience	Description

Online User Guide	Web page (HTML)	Participants, Uploaders	Step-by-step tutorials with screenshots covering registration, taking a quiz, creating a quiz, and viewing leaderboards.
FAQ Page	Static HTML page	All users	Answers to 15-20 anticipated common questions (e.g., “I forgot my password”, “Why is my file rejected?”).
In-App Tooltips	React component overlays	First-time users	Short contextual help bubbles on first visit to critical pages (e.g., “Click here to start a quiz”).
Administrator Handbook	PDF document	Administrators	Internal operations manual describing user management, content moderation workflow, and system monitoring.
API Documentation	Swagger / OpenAPI	Developers (internal)	Interactive documentation for all REST endpoints (not exposed publicly).

2.7 Assumptions and Dependencies

2.7.1 Assumptions

- **AS-01:** Users have access to a device with a modern browser and a stable internet connection.
- **AS-02:** Uploaders will only upload content they have the right to use; GameGuessr will rely on a “report abuse” mechanism rather than proactive copyright scanning during MVP.
- **AS-03:** The MySQL database will be available 99.9% of the time during operational hours. Backup and recovery procedures will be documented separately.
- **AS-04:** Peak concurrent users will not exceed **100** during MVP. Load testing will be performed prior to release to validate this assumption.
- **AS-05:** The development team has proficiency in JavaScript, React, Node.js, and MySQL.
- **AS-06:** The hosting environment will provide a static public IP address and domain name (e.g., GameGuessr.app).

2.7.2 Dependencies

Dependency	Version	Purpose	Impact if Unavailable
Socket.io	4.7.2	Real-time communication	Quiz Module becomes non-functional; fallback to polling degrades UX.
React	18.2.0	Frontend UI rendering	Application cannot be built or run.

Express	4.18.2	REST API framework	Backend API unavailable.
MySQL	8.0.35	Persistent data storage	No user data, quizzes, or scores can be saved or retrieved.
bcrypt	5.1.1	Password hashing	Security vulnerability; plaintext passwords would be unacceptable.
jsonwebtoken	9.0.2	JWT generation and verification	Authentication breaks; users cannot log in.
Multer	1.4.5	File upload handling	Upload Module cannot accept files.
pdf-parse	1.1.1	Extract text from PDF uploads	PDF uploads fail to generate questions.
mammoth	1.6.0	Extract text from DOCX uploads	DOCX uploads fail to generate questions.
Nginx	1.24.0	Reverse proxy and SSL termination	Production deployment security and routing would be suboptimal.

3. External Interface Requirements

3.1 User Interfaces

The GameGuessr UI is a responsive, mobile-first web application built with React and Material-UI (MUI). All pages share a common layout structure.

3.1.1 Common Layout Elements

Element	Description
Header	Contains the GameGuessr logo (links to home), main navigation (Quizzes, Leaderboard, Create Quiz if Uploader), and a user avatar dropdown (Profile, My History, Logout).
Footer	Contains links to About, FAQ, Terms of Service, Privacy Policy, and Contact.
Notification Area	A Snackbar (MUI) component that displays success, error, info, and warning messages for 5 seconds. All system feedback appears here.
Loading Spinner	A circular progress indicator displayed during API calls or WebSocket connection establishment.

3.1.2 Page-Specific UI Specifications

A. Landing Page (Guest View)

- **Hero Section:** Headline “Turn Knowledge into Competition”, sub-headline “Create quizzes from your content and challenge others in real time.”, and two buttons: “Get Started” (links to Sign Up) and “Browse Quizzes”.
- **Feature Highlights:** Three cards describing (1) Upload & Create, (2) Compete in Real-Time, (3) Climb the Leaderboard.
- **Preview Leaderboard:** A static, sample leaderboard showing fake usernames and scores to illustrate the competitive aspect.

B. Registration / Login Page

- **Form Fields:**
 - Registration: Username (alphanumeric, 3-20 chars), Email (valid format), Password (min 8 chars, must include at least one number and one letter), Confirm Password.
 - Login: Email, Password.
- **Validation:** Real-time client-side validation with error messages below each field. Server-side validation repeated for security.
- **Buttons:** “Sign Up” / “Log In”, plus a toggle link to switch between forms.

C. Participant Dashboard

- **Tab Navigation:** “Available Quizzes” (default) and “My Recent Activity”.
- **Quiz Cards:** Each quiz displays title, category tags, number of questions, average time to complete, and a “Start Quiz” button. Cards are sorted by most recently added.
- **My Recent Activity:** A list of the last 5 quizzes taken, showing title, score, date, and a “View Leaderboard” link.

D. Quiz Participation Interface

- **Header Bar:** Displays Quiz Title, current question number (e.g., “Question 5/10”), and a prominent countdown timer (format MM:SS).
- **Question Area:** Large, readable text for the question stem.
- **Answer Options:** Four radio-button style cards (A, B, C, D). Only one may be selected.
- **Action Button:** “Submit Answer”. This button is disabled until an option is selected. Once submitted, the button changes to “Next Question” (or “Finish Quiz” on the last question).
- **Feedback Overlay:** After submitting, a green (correct) or red (incorrect) banner appears for 1.5 seconds showing the result before automatically advancing to the next question.
- **Disconnection Handling:** If WebSocket disconnects, a modal overlay appears: “Connection lost. Attempting to reconnect...” with a spinner. When reconnected, the quiz resumes from the last unanswered question.

E. Quiz Creation Interface (Uploader)

- **Step 1 – Upload:** Drag-and-drop zone for file upload, or “Browse” button. Supported formats displayed. Progress bar during upload.
- **Step 2 – Configuration:** Form with fields:
 - Quiz Title (required, max 100 characters)
 - Category/Tags (comma-separated)
 - Time per Question: Slider or dropdown (15, 30, 45, 60, 90, 120 seconds). Default 30s.
 - Number of Questions to Use (from parsed content): numeric input, max limited by number of extracted questions.
- **Step 3 – Question Editor:** A list of generated questions. Each row shows question text, four answer options, and a checkbox for “Correct Answer”. Uploaders can edit any field, add new question, or delete a question. A “Preview” button shows how the question will appear to participants.
- **Step 4 – Publish:** Summary of quiz settings. “Publish Quiz” button makes it live.

F. Leaderboard Page

- **Filters:** Dropdown to select a specific quiz.
- **Table:** Columns: Rank (1,2,3 with medal emojis 🥇 🥈 🥉), Username, Score, Time Taken (in seconds).
- **Personal Highlight:** The logged-in user's row is highlighted with a light blue background.

G. Admin Panel (Administrator Only)

- **Users Tab:** Table listing all users with columns: Username, Email, Role (dropdown to change), Join Date, Actions (Delete).
- **Quizzes Tab:** Table of all quizzes with columns: Title, Uploader, Created Date, Status (Active/Hidden), Actions (Hide/Unhide, Delete).
- **Reports Tab:** Simple charts (using Chart.js) showing:
 - New user registrations per day (last 30 days)
 - Quizzes created per week
 - Average quiz score distribution.

3.1.3 Error Message Standards

All error messages presented to the user shall:

- Be displayed in a Snackbar with a red background.
- Use plain, non-technical language (e.g., “The email or password you entered is incorrect.” instead of “Invalid credentials”).
- Auto-dismiss after 5 seconds, or remain until dismissed if the error is critical (e.g., server error).
- Include a unique error code (e.g., ERR-AUTH-001) that is logged but not shown to the user; this aids support.

3.1.4 GUI Style Guide

Typography: Font family "Inter", sans-serif. Headings use weight 600, body text weight 400.

Color Palette:

- Primary: #3f51b5 (Indigo)
- Secondary: #f50057 (Pink accent)
- Success: #4caf50
- Error: #f44336
- Background: #f5f5f5

Spacing: Base unit 8px; all padding/margin multiples of 8px.

3.2 Hardware Interfaces

GameGuessr does not interface directly with any specialized hardware peripherals. It relies entirely on standard input/output devices managed by the operating system and browser.

- **Input Devices:** Keyboard, mouse, touchscreen (on mobile devices).
- **Output Devices:** Display monitor/screen.
- **No Direct Hardware Control:** The application does not access device sensors (camera, GPS, accelerometer) or storage beyond what the browser sandbox permits (e.g., LocalStorage, IndexedDB for caching).

3.3 Software Interfaces

This section details the interfaces between GameGuessr and external software components or libraries.

3.3.1 Database Interface – MySQL

Attribute	Description
DBMS	MySQL 8.0.35
Connection Method	mysql2 Node.js driver (promise-based)
Connection Pool	Minimum 10 connections, maximum 50. Idle timeout 10000 ms.
Authentication	Username/password stored in environment variables. No hard-coded credentials.
Data Exchange	SQL queries and result sets. Parameterized queries used exclusively to prevent SQL injection.
Port	3306 (default, internal network only)

3.3.2 File System Interface

Attribute	Description
Storage Location	/var/www/GameGuessr/uploads/
File Naming	{UUID}_{original_filename} to prevent collisions.
Access	Uploaded files are served statically via Nginx from a protected route that requires authentication.
Cleanup	A cron job runs nightly to delete files older than 7 days that are not associated with a published quiz.

3.3.3 Third-Party Libraries (NPM)

All libraries are managed via package.json. Key runtime dependencies:

Library	Version	Interface Description
express	4.18.2	Provides HTTP server and middleware routing.
socket.io	4.7.2	Attaches to the HTTP server to provide WebSocket functionality. Exports io object for event handling.

jsonwebtoken	9.0.2	jwt.sign() and jwt.verify() used for token management.
bcrypt	5.1.1	bcrypt.hash() and bcrypt.compare() for password handling.
multer	1.4.5	Middleware for handling multipart/form-data file uploads.
pdf-parse	1.1.1	Function pdf(dataBuffer) returns a promise with text content.
mammoth	1.6.0	mammoth.extractRawText({ buffer }) returns a promise with text.
express-validator	7.0.1	Middleware for validating and sanitizing request bodies.
helmet	7.1.0	Sets various HTTP headers for security.
cors	2.8.5	Configures Cross-Origin Resource Sharing for API endpoints.

3.3.4 Frontend Dependencies

Library	Version	Interface Description
react	18.2.0	Core UI library.
react-router-dom	6.20.0	Client-side routing.
@mui/material	5.14.0	UI component library implementing Google's Material Design.
axios	1.6.0	Promise-based HTTP client for REST API calls.
socket.io-client	4.7.2	WebSocket client that pairs with the server Socket.io.
chart.js	4.4.0	(Admin panel only) Rendering simple charts.

3.4 Communications Interfaces

3.4.1 REST API over HTTPS

- **Protocol:** HTTPS (TLS 1.2 or higher).
- **Base URL:** `https://api.GameGuessr.app/v1`
- **Authentication:** All endpoints except `/auth/register` and `/auth/login` require a valid JWT in the Authorization: Bearer <token> header.
- **Request Format:** JSON (Content-Type: application/json).
- **Response Format:** JSON with the following envelope structure:

```

json
{
  "status": "success",
  "data": { ... },

```

```

    "message": "Operation completed successfully."
  }
  or for errors:
  json
  {
    "status": "error",
    "error": {
      "code": "AUTH_002",
      "message": "Invalid email or password."
    }
  }
}

```

- **Rate Limiting:** Implemented using express-rate-limit. Default: 100 requests per IP per 15 minutes. Stricter limit on auth endpoints: 5 requests per minute.

3.4.2 WebSocket Communication (Socket.io)

- **Protocol:** WSS (WebSocket Secure) over the same port as HTTPS (443).
- **Connection URL:** wss://api.GameGuessr.app/socket.io/
- **Authentication:** The Socket.io connection is authenticated using the same JWT sent as a query parameter or in the auth handshake object.
- **Namespaces / Rooms:**
 - Each quiz session is mapped to a **Socket.io room** named quiz: {quizId}:{sessionId}.
 - Participants join the room upon starting a quiz.
 - The server emits events to the room to synchronize timers and advance questions.
- **Event Schema (Server → Client):**
 - question:new – payload: { questionId, text, options, timeLimit, questionNumber, totalQuestions }
 - timer:sync – payload: { remainingSeconds } (sent every second)
 - answer:result – payload: { correct, correctAnswerId, scoreDelta }
 - quiz:completed – payload: { finalScore, totalQuestions, leaderboardRank }
 - error – payload: { code, message }
- **Event Schema (Client → Server):**
 - answer:submit – payload: { questionId, answerId }
 - quiz:leave – payload: { } (graceful disconnect)

3.4.3 Email Communication (Optional for MVP)

- **Purpose:** Send account verification and password reset emails.
- **Implementation:** If implemented, the system will use **Nodemailer** with an SMTP relay (e.g., SendGrid, AWS SES).
- **Message Format:** HTML emails with a plain-text fallback.
- **Security:** All links in emails will use HTTPS and include a time-limited, single-use token.

4. System Features

This section organizes functional requirements by major features. Each requirement is uniquely identified and prioritized.

4.1 Authentication and Authorization

4.1.1 Description and Priority

This feature manages user identity, session management, and access control based on roles. It is the foundational security component of GameGuessr.

Priority: High – Without it, no other features can function securely.

4.1.2 Stimulus/Response Sequences

Actor	Action	System Response
Guest	Submits registration form with valid data.	1. Validates input. 2. Checks email uniqueness. 3. Hashes password. 4. Inserts user record with role 'participant'. 5. Returns 201 Created with user info (excluding password). 6. Optionally sends verification email.
Guest	Submits registration with an already registered email.	Returns 400 Bad Request with error EMAIL_EXISTS.
User	Submits login form with valid credentials.	1. Finds user by email. 2. Compares password with bcrypt. 3. Generates JWT (expires in 24h). 4. Returns 200 OK with token and user profile.
User	Submits login with invalid credentials.	Returns 401 Unauthorized with error INVALID_CREDENTIALS.
User	Logs out.	Client discards JWT. Server does not maintain state; token is simply not used further.
Participant	Attempts to access /admin/users endpoint.	Server middleware checks role claim in JWT. If role !== 'admin', returns 403 Forbidden.
Admin	Promotes a user to Uploader via admin panel.	1. Validates admin token. 2. Updates user's role in database to 'uploader'. 3. Returns 200 OK.

4.1.3 Functional Requirements

- **REQ-AUTH-001 [High]:** The system shall provide an endpoint POST /auth/register that accepts username, email, and password in the request body.
- **REQ-AUTH-002 [High]:** The system shall enforce that username is unique, alphanumeric, and between 3 and 20 characters.

- **REQ-AUTH-003 [High]:** The system shall enforce that email is a valid email format and unique.
- **REQ-AUTH-004 [High]:** The system shall enforce that password is at least 8 characters long and contains at least one letter and one number.
- **REQ-AUTH-005 [High]:** The system shall hash all user passwords using bcrypt with a salt factor of 10 before storage.
- **REQ-AUTH-006 [High]:** The system shall provide an endpoint POST /auth/login that accepts email and password and returns a signed JWT upon successful authentication.
- **REQ-AUTH-007 [High]:** The JWT payload shall include userId, username, and role. The token shall expire after 24 hours.
- **REQ-AUTH-008 [High]:** All protected API endpoints shall require a valid JWT in the Authorization header. Missing or invalid tokens shall result in 401 Unauthorized.
- **REQ-AUTH-009 [Medium]:** The system shall provide an endpoint POST /auth/refresh that accepts a valid (but not expired) JWT and returns a new token with a fresh 24-hour expiry.
- **REQ-AUTH-010 [Medium]:** The system shall log all authentication events (login success/failure, registration, role change) to a audit_log table with timestamp and IP address.
- **REQ-AUTH-011 [High]:** The system shall implement role-based middleware that restricts access to endpoints:
 - requireAuth – any authenticated user.
 - requireUploader – role must be 'uploader' or 'admin'.
 - requireAdmin – role must be 'admin'.

4.2 Uploader Module - Content Upload and Quiz Configuration

4.2.1 Description and Priority

This feature enables users with the Uploader role to submit content files, have them parsed into quiz questions, configure quiz parameters, and publish the quiz for participants.

Priority: High – Content is the fuel for the platform.

4.2.2 Stimulus/Response Sequences

Actor	Action	System Response
Uploader	Navigates to /create and uploads a valid .txt file (2 MB).	1. Saves file temporarily. 2. Extracts text. 3. Calls question-generation algorithm. 4. Returns a list of generated questions (max 20) to the frontend editor.
Uploader	Uploads a .pdf file with scanned image (no text layer).	1. Attempts text extraction, receives empty string. 2. Returns error: “Could not extract text. Please ensure the PDF contains selectable text.”
Uploader	Uploads a file larger than 10 MB.	1. Multer rejects file. 2. Returns 413 Payload Too Large.

Uploader	Edits a generated question's answer options.	Frontend updates state locally. Changes are not persisted until "Publish" is clicked.
Uploader	Clicks "Publish Quiz" after configuring all fields.	1. Validates that title is present and at least one question exists. 2. Saves quiz metadata and questions to database. 3. Returns 201 Created with quizId. 4. Makes quiz discoverable.
Uploader	Attempts to publish a quiz with no questions.	Returns 400 Bad Request: "A quiz must have at least one question."

4.2.3 Functional Requirements

File Upload & Parsing

- **REQ-UPLOAD-001 [High]:** The system shall accept file uploads via POST /quizzes/upload with multipart/form-data. Supported MIME types: text/plain, application/pdf, application/vnd.openxmlformats-officedocument.wordprocessingml.document, application/json.
- **REQ-UPLOAD-002 [High]:** The system shall enforce a maximum file size of 10 MB. Requests exceeding this shall receive HTTP 413.
- **REQ-UPLOAD-003 [High]:** For .txt and extracted text from PDF/DOCX, the system shall apply a question-generation algorithm that:
 - Splits text into sentences.
 - Identifies key noun phrases as candidate answers.
 - Constructs a multiple-choice question with three distractors.
 - Returns a maximum of 20 questions per upload.
- **REQ-UPLOAD-004 [Medium]:** The system shall allow Uploaders to upload a JSON file that directly defines questions and answers in a predefined schema (see Appendix B for JSON schema).

Quiz Configuration

- **REQ-UPLOAD-005 [High]:** The system shall provide an endpoint POST /quizzes that accepts a JSON body containing:
 - title (string, required, max 100)
 - description (string, optional, max 500)
 - categoryTags (array of strings, optional)
 - timePerQuestion (integer seconds, required, 15–120)
 - questions (array of question objects, required)
- **REQ-UPLOAD-006 [High]:** Each question object shall contain:
 - text (string, required)
 - options (array of exactly 4 strings, required)
 - correctOptionIndex (integer 0-3, required)
- **REQ-UPLOAD-007 [Medium]:** The system shall allow the Uploader to preview the quiz in a modal that simulates the participant view before publishing.
- **REQ-UPLOAD-008 [High]:** Upon successful quiz creation, the system shall store the quiz record with status = 'published' and created_by = userId of the Uploader.

- **REQ-UPLOAD-009 [Medium]:** The system shall allow the Uploader to edit a quiz they created **only if** no participant has yet taken it. If the quiz has been taken, edits are disallowed and a message “Cannot edit quiz that has already been played” is shown.
- **REQ-UPLOAD-010 [Low]:** The system shall allow an Uploader to delete a quiz they own, provided it has no scores recorded. If scores exist, the quiz is soft-deleted (status = 'archived').

4.3 Quiz Module - Real-Time Quiz Session Management

4.3.1 Description and Priority

This feature delivers timed, sequential questions to participants, records answers in real time, calculates scores, and manages the lifecycle of a quiz session.

Priority: High – This is the core interactive experience.

4.3.2 Stimulus/Response Sequences (Detailed)

Actor	Action	System Response
Participant	Click “Start Quiz” on a quiz card.	1. Client establishes Socket.io connection with auth token. 2. Client emits quiz:join with quizId. 3. Server creates or joins a room quiz:{quizId}:{sessionId}. 4. Server emits quiz:started and first question:new.
Participant	Selects option “B” and clicks “Submit Answer” with 10 seconds left.	1. Client emits answer:submit with questionId and answerId. 2. Server validates that the answer is for the current question. 3. Server records answer, updates score. 4. Server emits an answer:result to that client only. 5. Server advances questions for that client after a 1.5-second delay.
Participant	Does not submit before the timer expires.	1. Server automatically emits an answer:result with correct: false and scoreDelta: 0. 2. Server advances to the next question.
Participant	Complete final question.	1. The server calculates the final score. 2. Server saves score to database. 3. Server emits quiz:completed with final score and rank. 4. Client leaves room.
Participant	Loses internet connection during quiz.	1. Socket.io client attempts reconnection with exponential backoff. 2. If reconnected within 60 seconds, server identifies the session and resumes from the next unanswered question. 3. If reconnection fails, session is terminated and no score is recorded.

System	Server-side timer for a question.	The server maintains a master timer for each participant. Every second, it broadcasts timer:sync to the client. This prevents client-side timer manipulation.
---------------	-----------------------------------	---

4.3.3 Functional Requirements

Session Initiation

- **REQ-QUIZ-001 [High]:** The system shall allow an authenticated participant to join a quiz session via a WebSocket connection authenticated with the user's JWT.
- **REQ-QUIZ-002 [High]:** The system shall assign each quiz attempt a unique sessionId and create a corresponding record in the quiz_sessions table.
- **REQ-QUIZ-003 [High]:** The system shall retrieve questions for the quiz in a **random order** unique to each participant to reduce cheating.

Real-Time Question Flow

- **REQ-QUIZ-004 [High]:** The server shall emit a question:new event containing the question text, four answer options, the time limit for that question, and the current question number.
- **REQ-QUIZ-005 [High]:** The server shall maintain a per-participant countdown timer. Every second, it shall emit a timer:sync event with remainingSeconds.
- **REQ-QUIZ-006 [High]:** When the timer reaches zero without an answer, the server shall automatically treat the answer as incorrect and advance to the next question.
- **REQ-QUIZ-007 [High]:** Upon receiving an answer:submit event, the server shall:
 - Validate that the questionId matches the currently active question for that participant.
 - Compare the submitted answerId with the correct option ID.
 - If correct, increment the participant's running score by **10 points**.
 - Record the response in the responses table with timestamp.
 - Emit answer:result to the client.
 - Schedule the next question emission after a fixed **1.5-second** delay.

Scoring and Completion

- **REQ-QUIZ-008 [High]:** The system shall calculate the final score as the sum of points for correct answers. Partial credit is not awarded.
- **REQ-QUIZ-009 [High]:** Upon answering the last question (or timer expiry), the system shall:
 - Save the final score and completion time to the scores table.
 - Emit a quiz:completed event to the client containing finalScore, totalQuestions, and the participant's current rank on the leaderboard.
 - Close the WebSocket room for that participant.

Error Handling and Resilience

- **REQ-QUIZ-010 [High]:** If a participant disconnects during a quiz, the server shall retain the session state for **60 seconds**. If the client reconnects with the same sessionId, the server shall resume from the next unanswered question.
- **REQ-QUIZ-011 [High]:** The system shall prevent a participant from joining a quiz session more than once concurrently.
- **REQ-QUIZ-012 [Medium]:** The system shall log all real-time events (question start, answer submission, disconnection) for debugging and analytics.

4.4 Leaderboard Module – Scoring and Ranking

4.4.1 Description and Priority

This feature computes and displays rankings for each quiz and provides participants with historical performance data.

Priority: High – Essential for the competitive, gamified experience.

4.4.2 Stimulus/Response Sequences

Actor	Action	System Response
Participant	Views leaderboard for Quiz ID 123.	1. GET /leaderboard/:quizId returns top 50 scores ordered by score descending, then by time ascending.
Participant	Views “My History” page.	1. GET /users/me/history returns list of past quiz attempts with score, date, and rank.
Admin	Requests global report.	1. GET /admin/reports/usage returns JSON with aggregated stats.

4.4.3 Functional Requirements

- **REQ-LEAD-001 [High]:** The system shall provide an endpoint GET /leaderboard/:quizId that returns a sorted list of participants. Each entry shall include: username, score, completedAt, and rank.
- **REQ-LEAD-002 [High]:** The leaderboard shall be sorted by score descending. In case of a tie, the participant with the lower completionTime (in seconds) shall rank higher.
- **REQ-LEAD-003 [Medium]:** The system shall support pagination for leaderboards when more than 50 entries exist (e.g., ?page=2&limit=50).
- **REQ-LEAD-004 [High]:** The system shall provide an endpoint GET /users/me/history that returns:
 - A list of quizzes taken by the authenticated user, including quizTitle, score, totalQuestions, date, and rank.
 - Summary statistics: averageScore, totalQuizzesTaken, highestScore.
- **REQ-LEAD-005 [Medium]:** The system shall cache leaderboard results for 60 seconds to reduce database load during high traffic.
- **REQ-LEAD-006 [Low]:** The system shall award a “Top 3” badge to users who finish in the top three positions on any leaderboard. This badge is displayed on the user’s profile.

4.5 Administrative Control Panel

4.5.1 Description and Priority

This feature provides system administrators with tools to manage users, moderate content, and monitor system health.

Priority: Medium – Not required for initial participant engagement, but critical for long-term operation.

4.5.2 Functional Requirements

User Management

- **REQ-ADMIN-001 [High]:** The system shall provide an endpoint GET /admin/users (admin only) that returns a paginated list of all users with filters by role and search by username/email.

- **REQ-ADMIN-002 [High]:** The system shall provide an endpoint PATCH /admin/users/:userId that allows an admin to update a user's role (to 'participant' or 'uploader').
- **REQ-ADMIN-003 [Medium]:** The system shall provide an endpoint DELETE /admin/users/:userId that soft-deletes a user account (sets isActive = false).

Content Moderation

- **REQ-ADMIN-004 [High]:** The system shall provide an endpoint GET /admin/quizzes with filters by status and uploader.
- **REQ-ADMIN-005 [High]:** The system shall allow an admin to hide a quiz via PATCH /admin/quizzes/:quizId with body { status: 'hidden' }. Hidden quizzes do not appear in participant dashboards but remain in the database.
- **REQ-ADMIN-006 [Medium]:** The system shall allow an admin to permanently delete a quiz and all associated scores via DELETE /admin/quizzes/:quizId.

Reporting

- **REQ-ADMIN-007 [Low]:** The system shall provide an endpoint GET /admin/stats that returns:
 - Total registered users
 - Total quizzes created
 - Total quiz attempts
 - Average score across all attempts
 - New users per day (last 7 days)
- **REQ-ADMIN-008 [Medium]:** All administrative actions shall be logged with adminId, action, targetType, targetId, and timestamp.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

ID	Requirement	Priority	Measurement Method
PERF-01	The landing page shall load and become interactive within 2.0 seconds on a 4G network (simulated).	High	Lighthouse performance score ≥ 90 , First Contentful Paint $< 1.5s$.
PERF-02	REST API response time (95th percentile) shall be ≤ 300 ms under a load of 50 concurrent users.	High	Apache Bench or k6 load test against /quizzes endpoint.
PERF-03	WebSocket question:new latency (server emit to client receipt) shall be ≤ 200 ms for 95% of events.	High	Instrumented client logging during load test.

PERF-04	The system shall support 100 concurrent WebSocket connections (active quiz participants) without exceeding 70% CPU utilization on the server.	High	Load test with Artillery or similar WebSocket testing tool.
PERF-05	Database queries for leaderboard retrieval (top 50) shall execute in ≤100 ms .	Medium	MySQL EXPLAIN and query profiling.

5.2 Safety Requirements

- **SAFE-01 [High]:** The system shall sanitize all user-generated content (quiz titles, questions, answer options, usernames) before rendering in HTML to prevent Cross-Site Scripting (XSS) attacks. React's default escaping is sufficient; dangerouslySetInnerHTML shall not be used with user content.
- **SAFE-02 [Medium]:** In the event of a server crash during a quiz session, the system shall recover by reading the quiz_sessions and responses tables to determine which participants were mid-quiz. Those participants shall be offered the option to resume or retake the quiz (with admin discretion).

5.3 Security Requirements

- **SEC-01 [High]:** All communication between client and server shall be encrypted using TLS 1.2 or higher. HTTP requests shall be redirected to HTTPS.
- **SEC-02 [High]:** Passwords shall never be logged or transmitted in plaintext. They shall be hashed using bcrypt (cost factor 10).
- **SEC-03 [High]:** JWTs shall be signed using the HS256 algorithm with a secret key of at least 256 bits, stored securely in environment variables.
- **SEC-04 [High]:** The application shall set secure HTTP headers using Helmet.js, including X-Content-Type-Options: nosniff, X-Frame-Options: DENY, and a strict Content Security Policy.
- **SEC-05 [High]:** All SQL queries shall use parameterized statements or an ORM with input escaping to prevent SQL injection.
- **SEC-06 [Medium]:** Rate limiting shall be applied to authentication endpoints: maximum 5 failed login attempts per IP per 15 minutes.
- **SEC-07 [Medium]:** File uploads shall be scanned for malware using ClamAV (optional) or restricted to non-executable MIME types. Uploaded files shall not be executable by the web server.

5.4 Software Quality Attributes

Attribute	Requirement	Verification Method
Usability	A first-time participant shall be able to register, find a quiz, and complete it within 5 minutes without external help.	Usability testing with 5 representative users.
Reliability	The system shall have an uptime of 99.5% during peak hours (9am-9pm) excluding scheduled maintenance.	Uptime monitoring tool (e.g., UptimeRobot).
Maintainability	The backend code shall follow the MVC pattern with clear separation of concerns. All public functions shall have JSDoc comments.	Code review checklist.
Scalability	Although MVP is single-server, the WebSocket layer shall be designed such that it can be extracted into a separate service using the Redis adapter for Socket.io in future phases.	Architecture review.
Testability	Each REST endpoint shall have at least one integration test covering success and failure scenarios.	Jest test suite coverage report (target >80%).

5.5 Business Rules

- **BR-01:** A user may only submit one score per quiz. If a user attempts to retake a quiz, the system shall prevent it and display “You have already completed this quiz.”
- **BR-02:** An Uploader may not participate in a quiz they created.
- **BR-03:** The default role for all new registered users is “Participant”.
- **BR-04:** A quiz with zero participants for 30 consecutive days shall be automatically archived (hidden).
- **BR-05:** The top three participants on any leaderboard shall receive a visual badge (🏆 🥈 🥉) on their profile.

6. Other Requirements

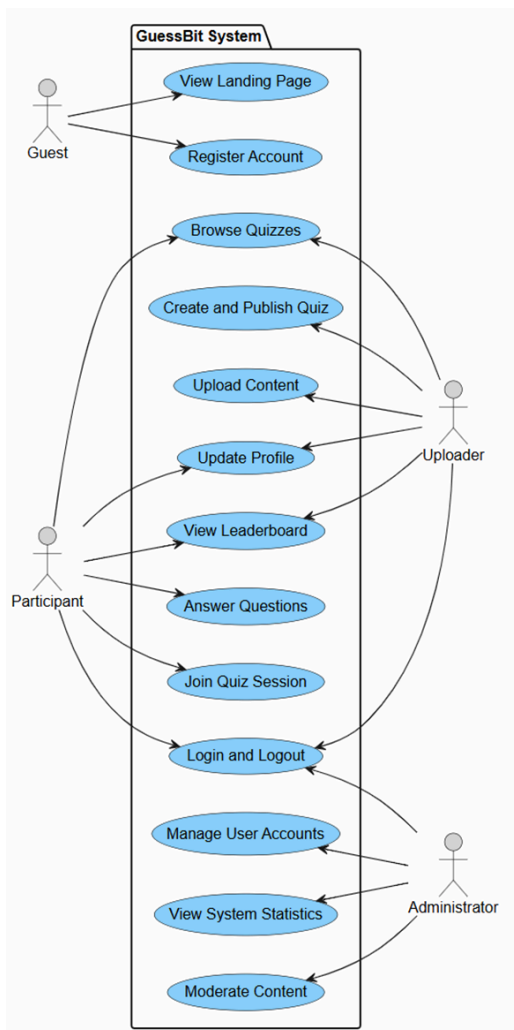
Appendix A: Glossary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

Appendix B: Analysis Models

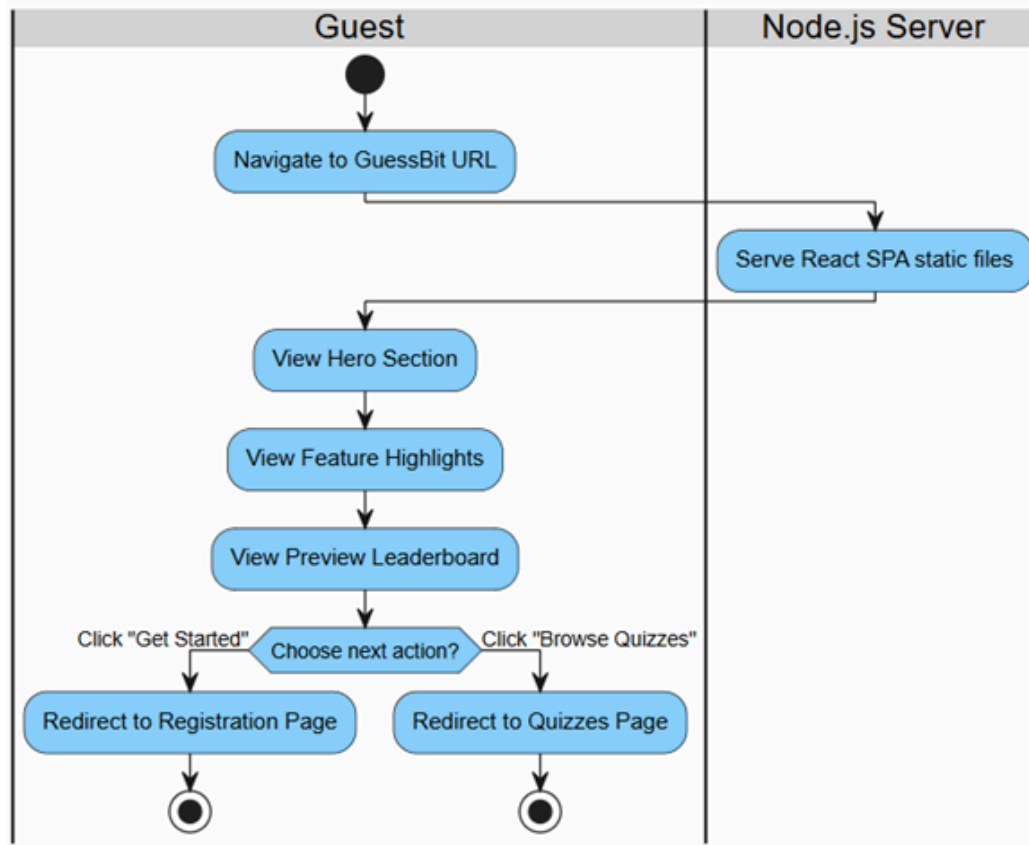
<Optionally, include any pertinent analysis models, such as data flow diagrams, class diagrams, state-transition diagrams, or entity-relationship diagrams.>

B.1 USE CASE DIAGRAM

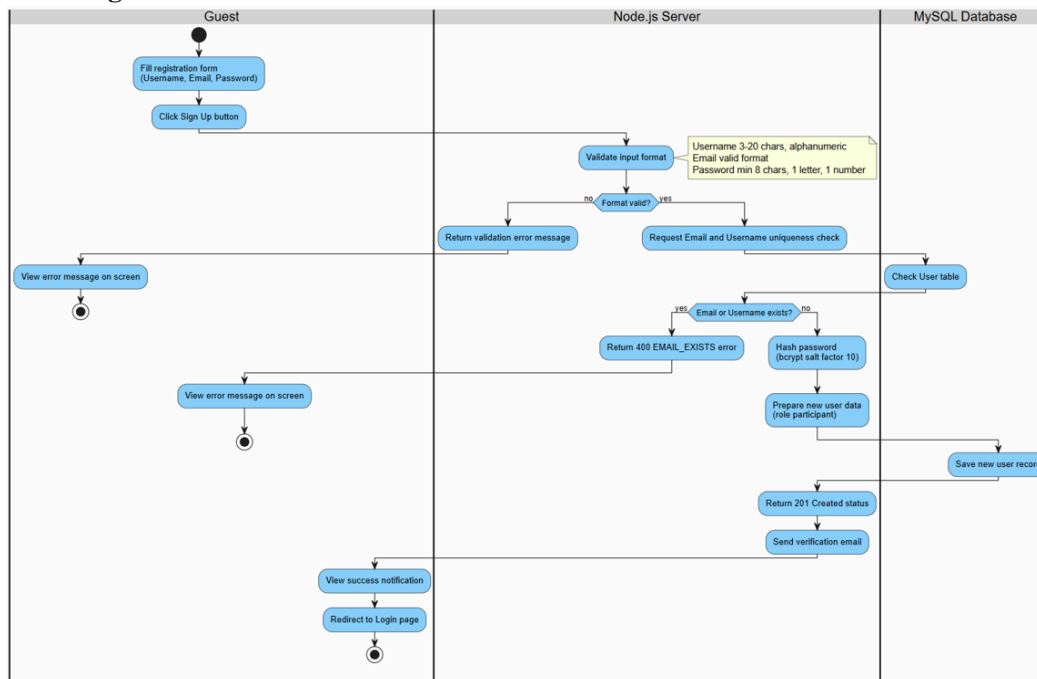


B.2 ACTIVITY DIAGRAM

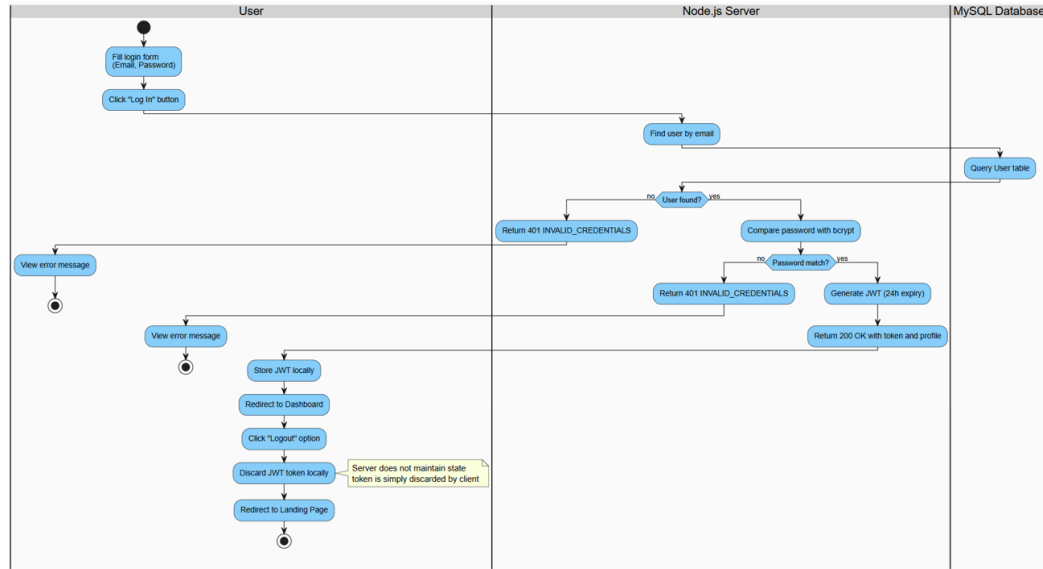
B.2.1 View Landing Page



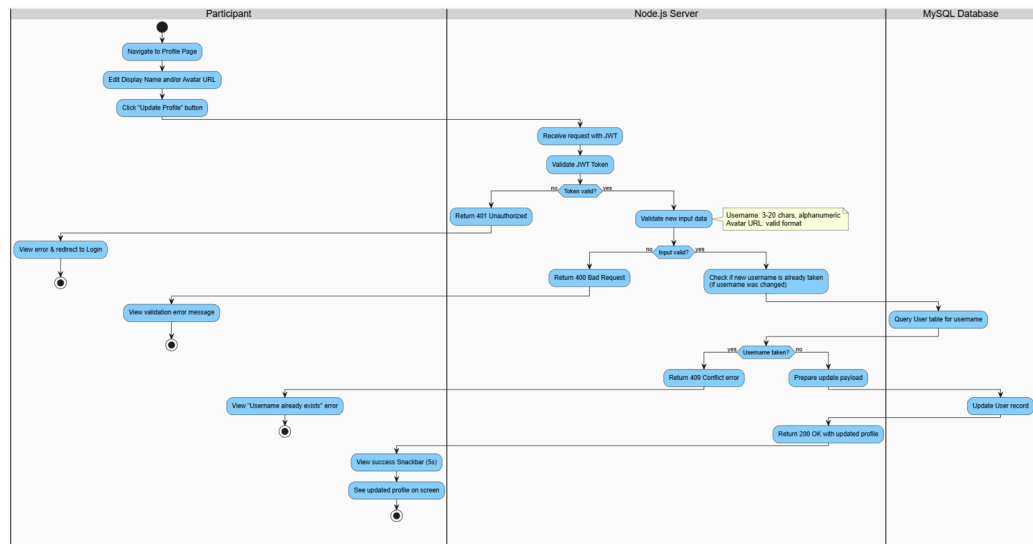
B.2.2 Register Account



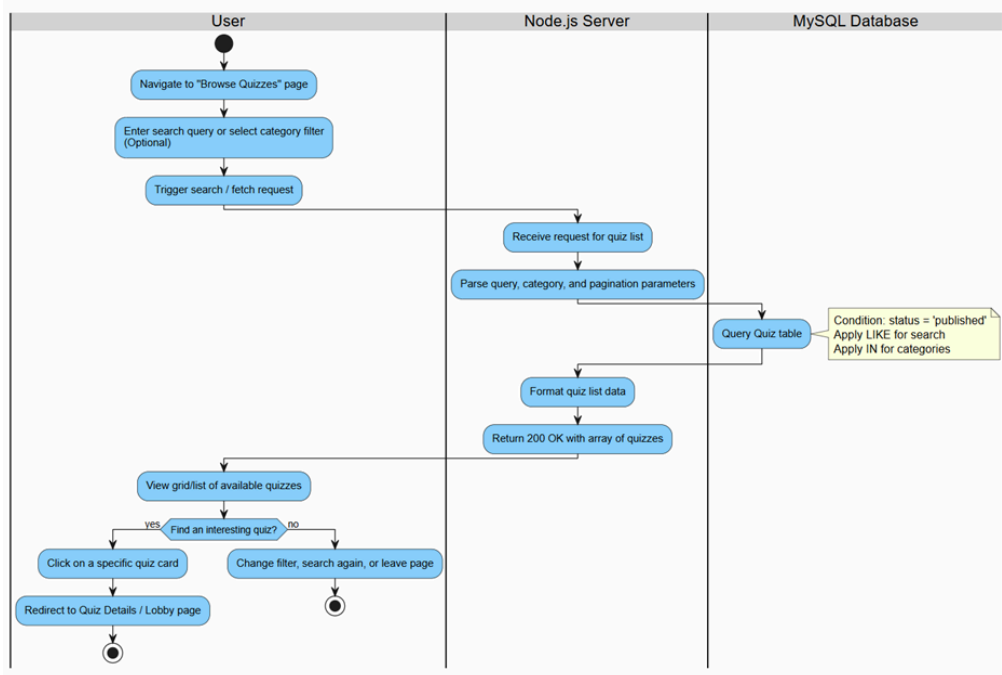
B.2.3 Login and Logout



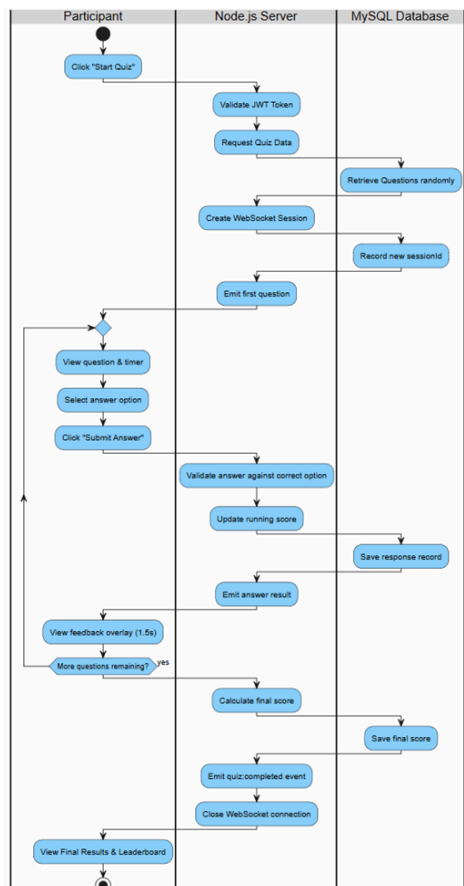
B.2.4 Update Profile



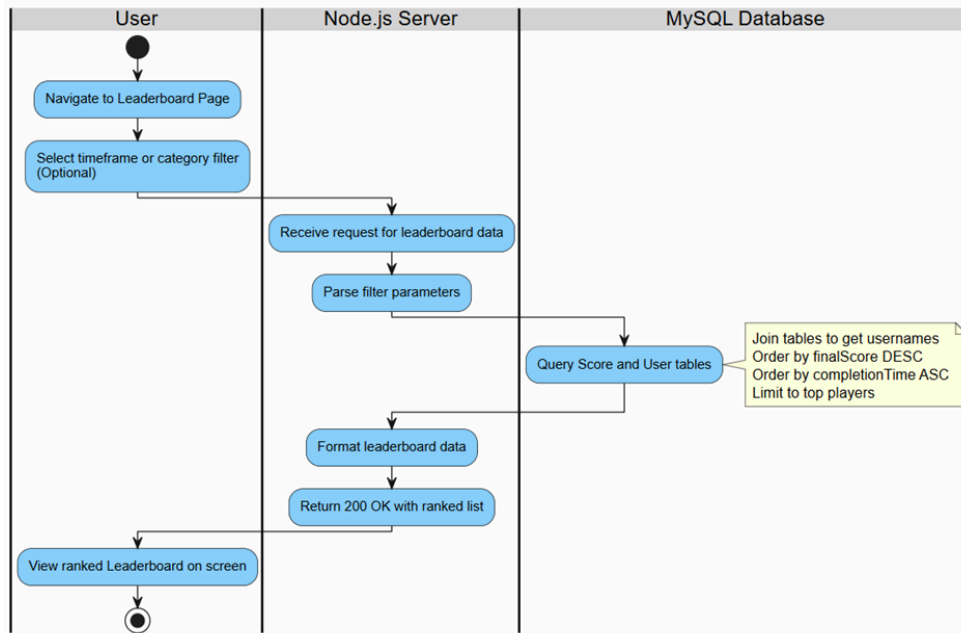
B.2.5 Browse Quizzes



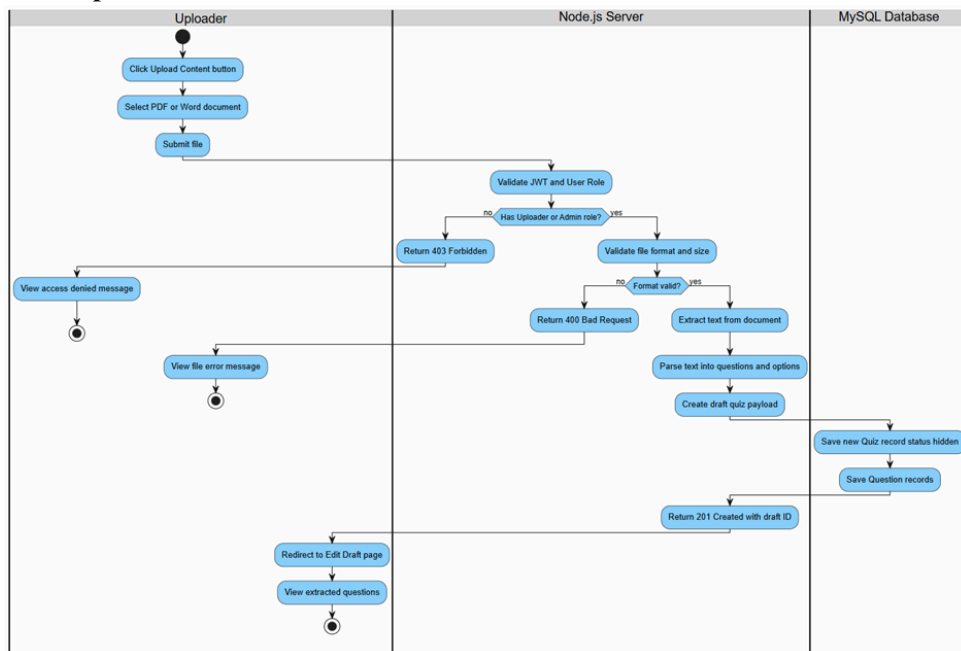
B.2.6 Join Quiz Session + Answer Questions



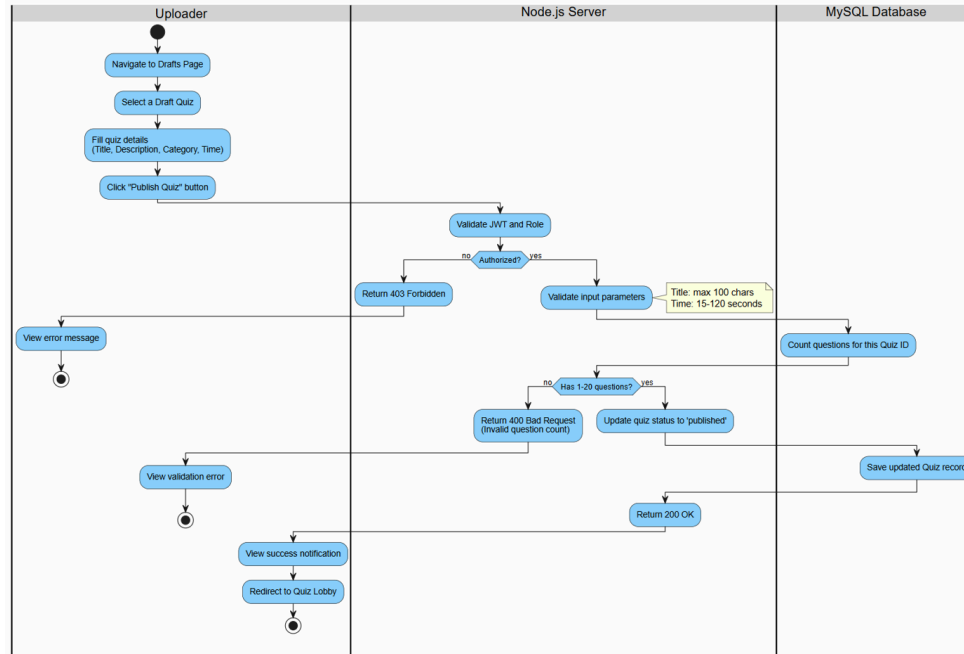
B.2.7 View Leaderboard



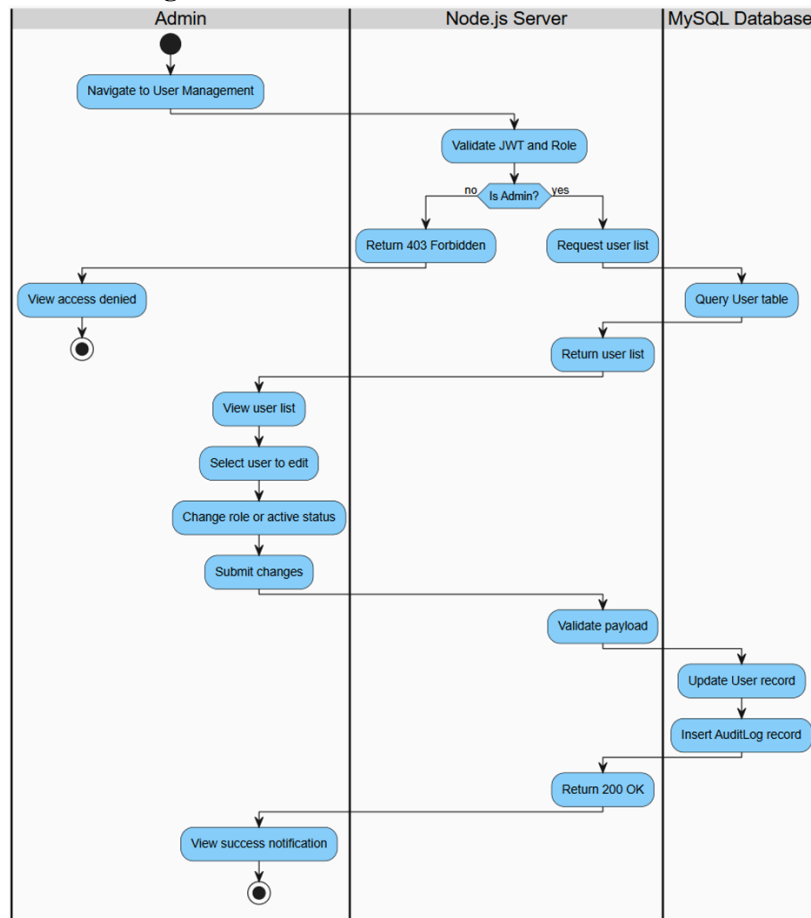
B.2.8 Upload Content



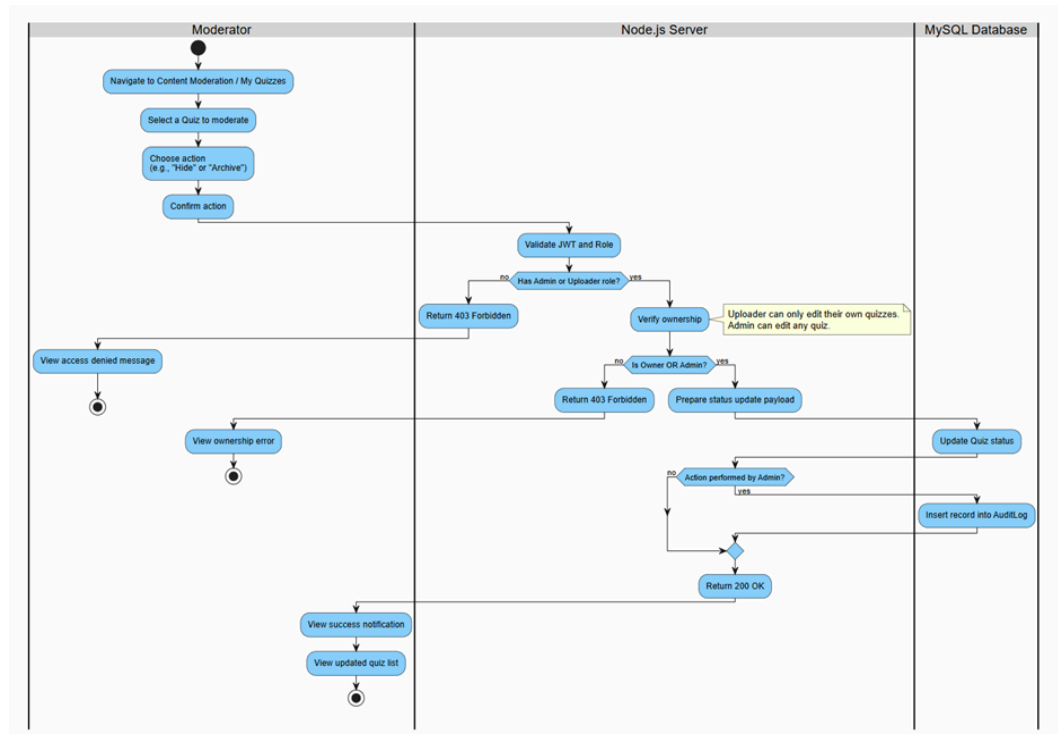
B.2.9 Create and Publish Quiz



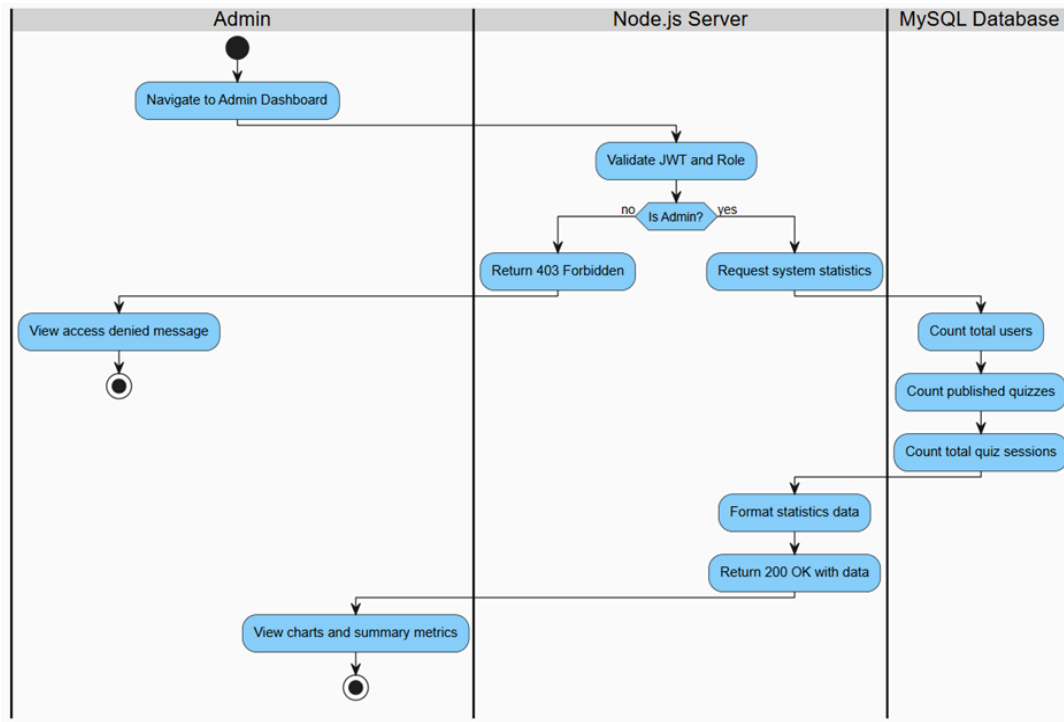
B.2.10 Manage User Accounts



B.2.11 Moderate Content

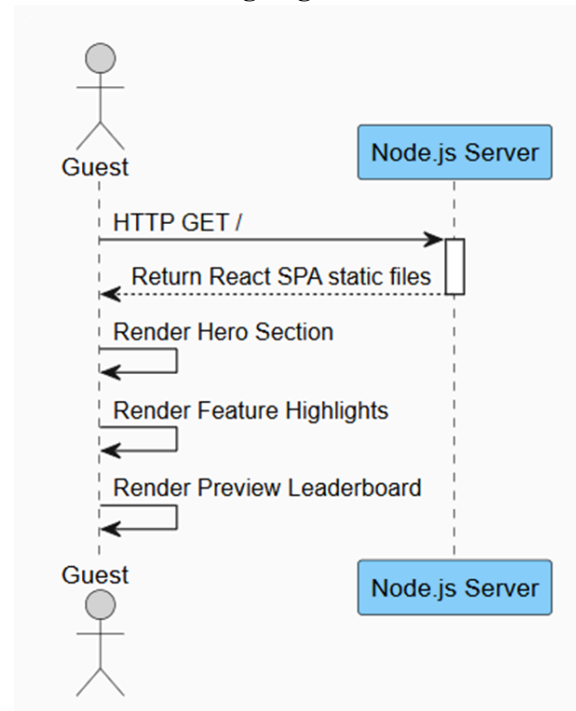


B.2.12 View System Statistics

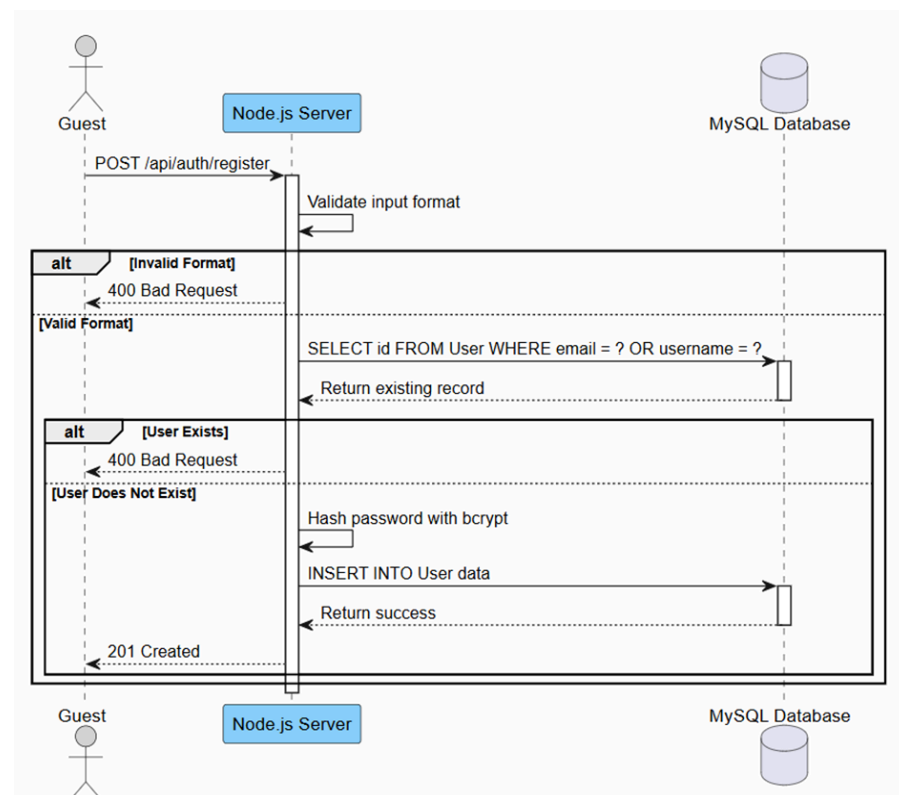


B.3 SEQUENCE DIAGRAM

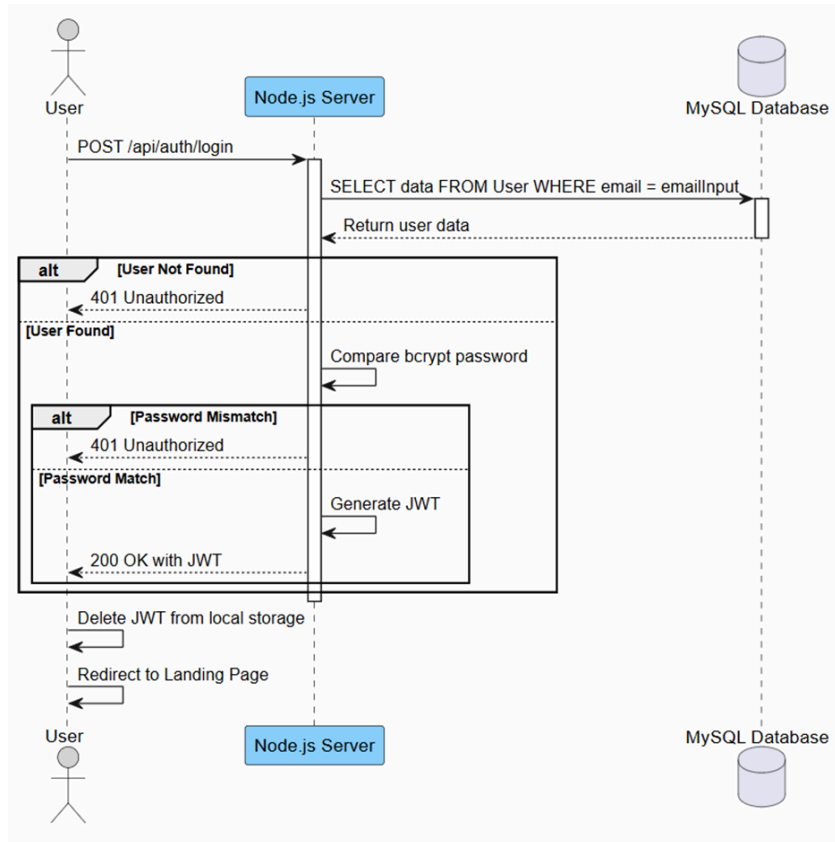
B.3.1 View Landing Page



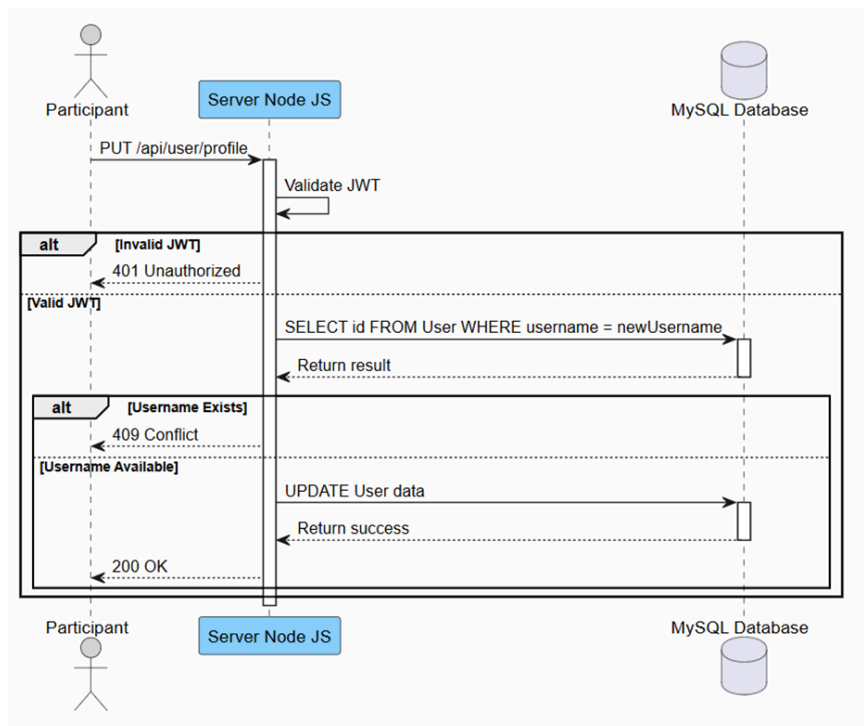
B.3.2 Register Account



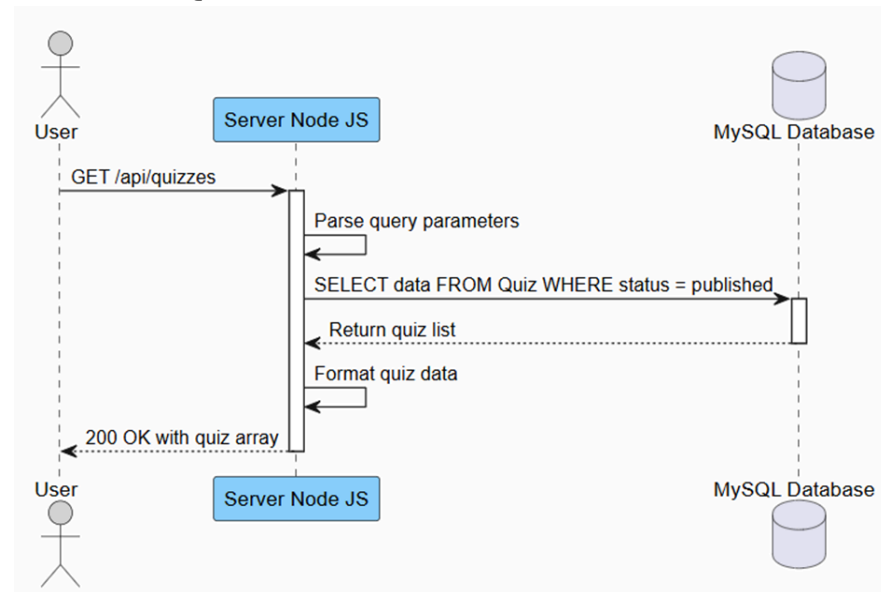
B.3.3 Login and Logout



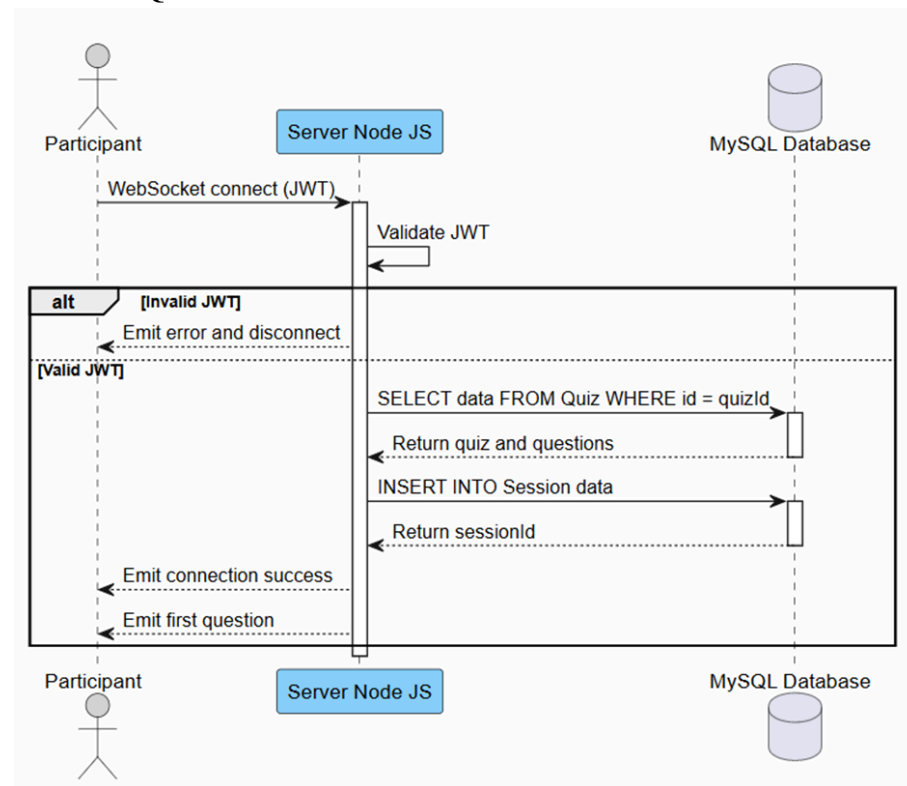
B.3.4 Update Profile



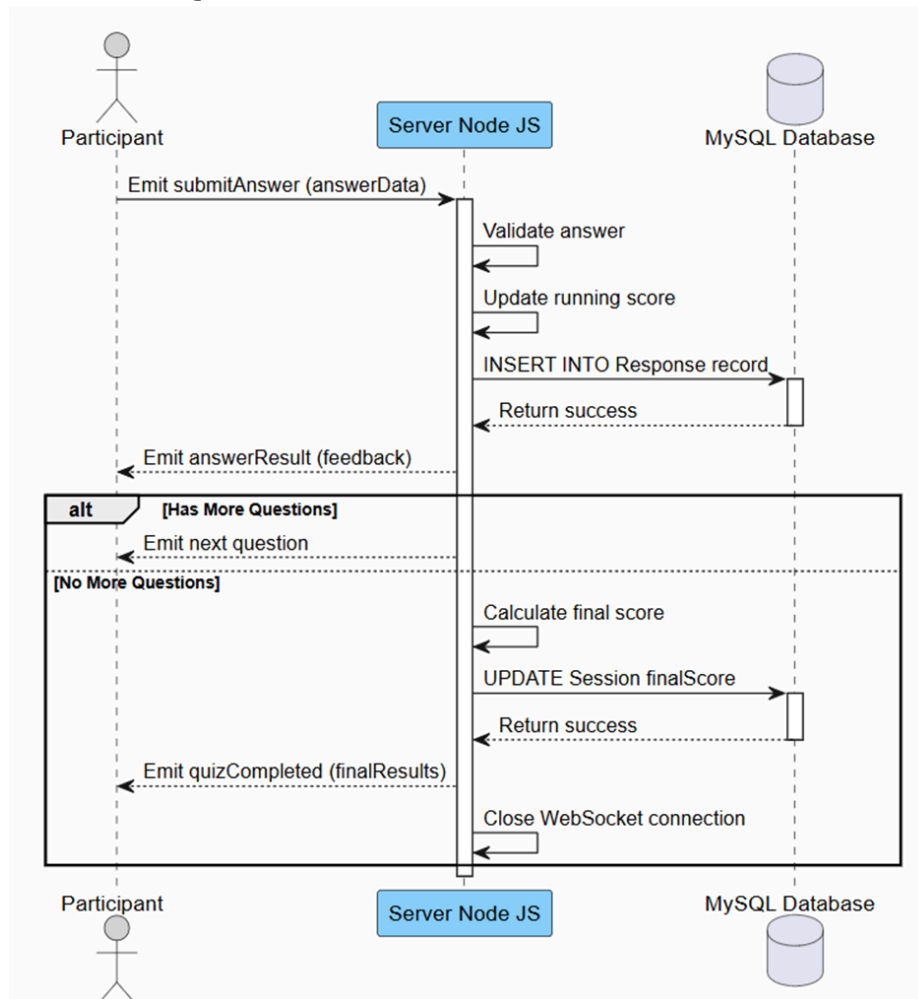
B.3.5 Browse Quizzes



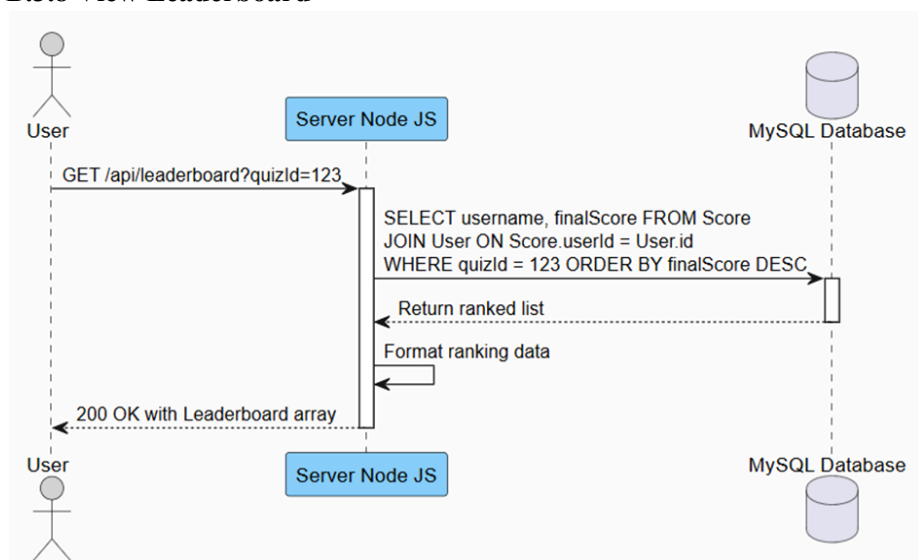
B.3.6 Join Quiz Session



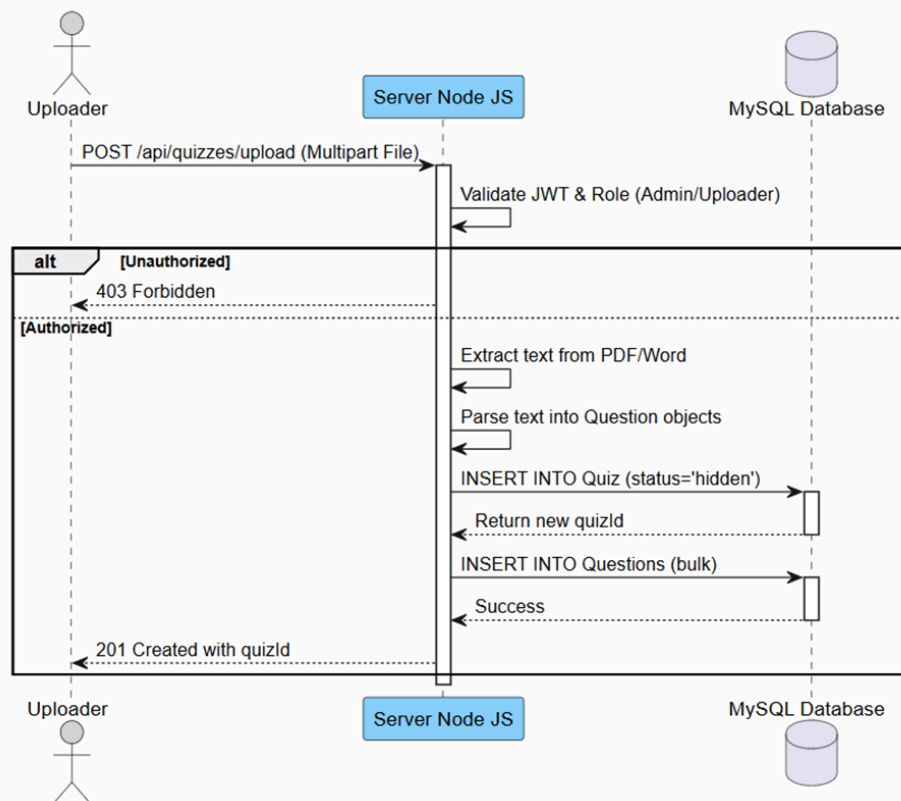
B.3.7 Answer Questions



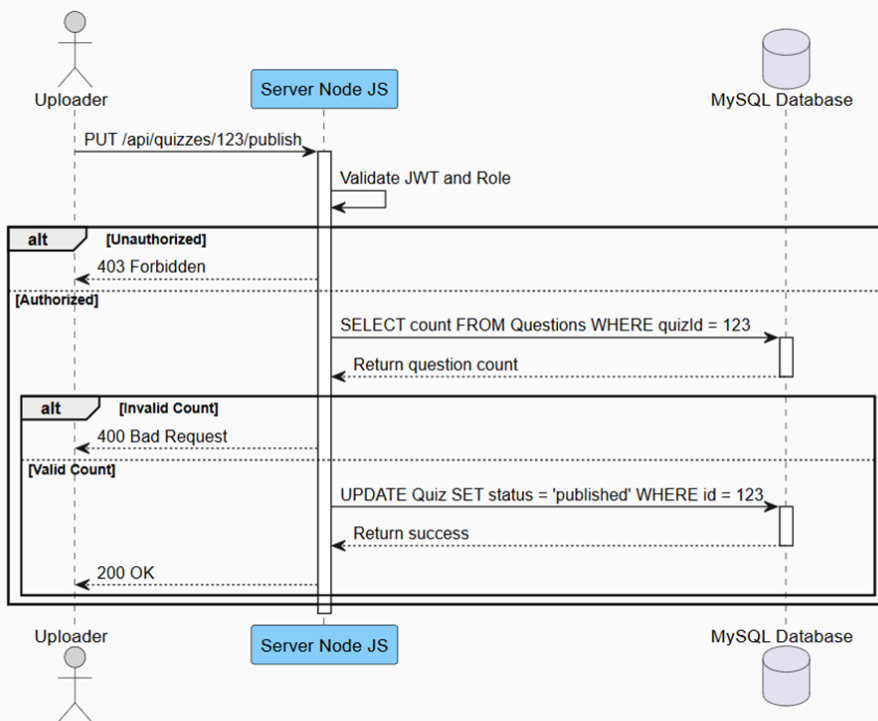
B.3.8 View Leaderboard



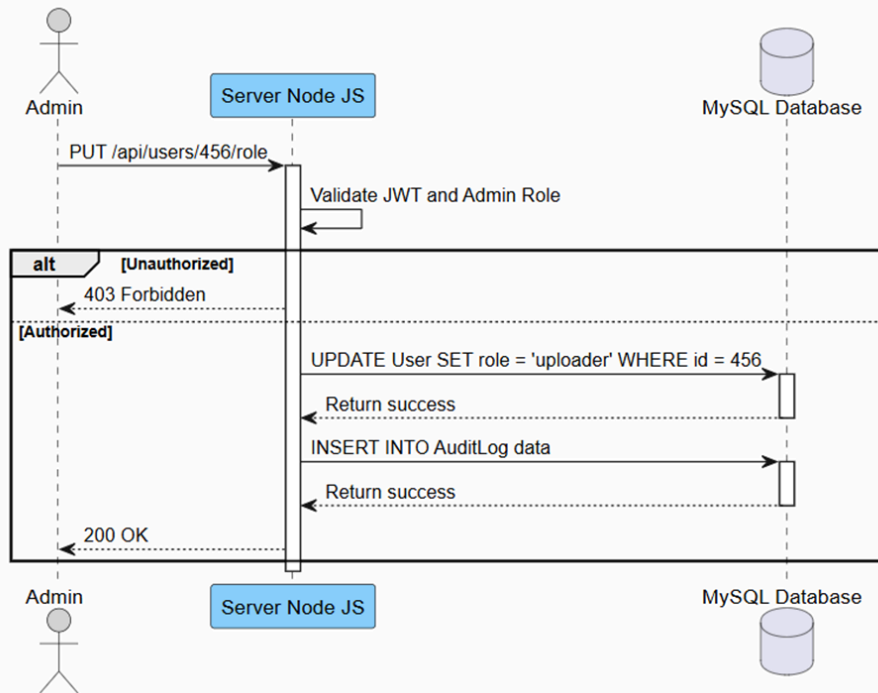
B.3.9 Upload Content



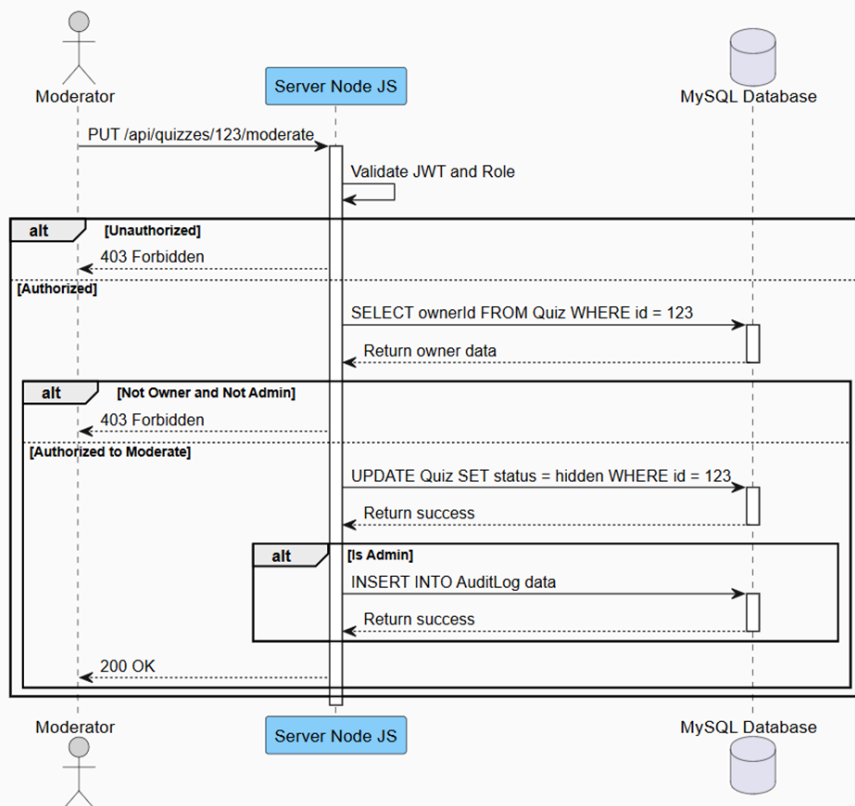
B.3.10 Create and Publish Quiz



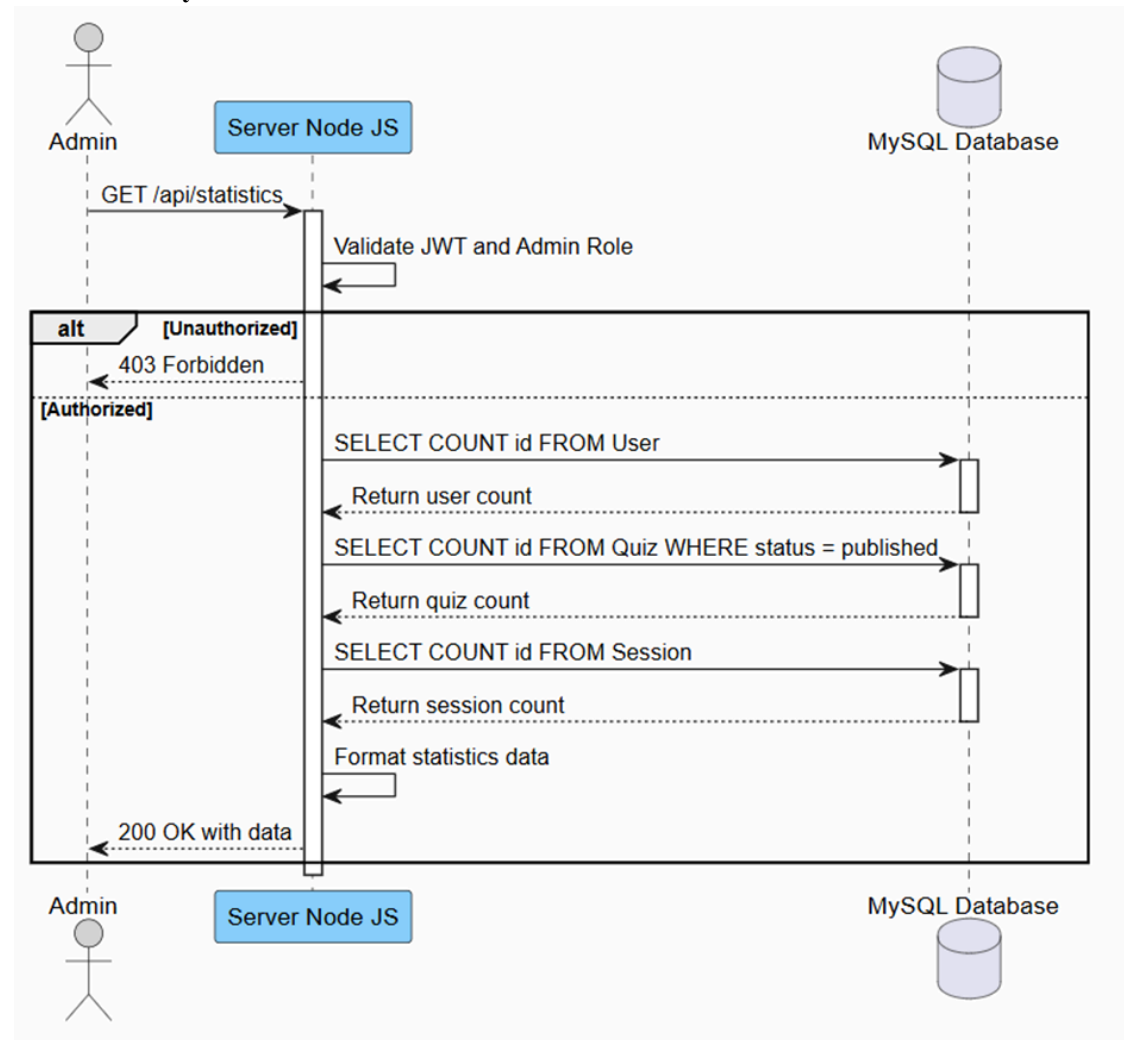
B.3.11 Manage User Accounts



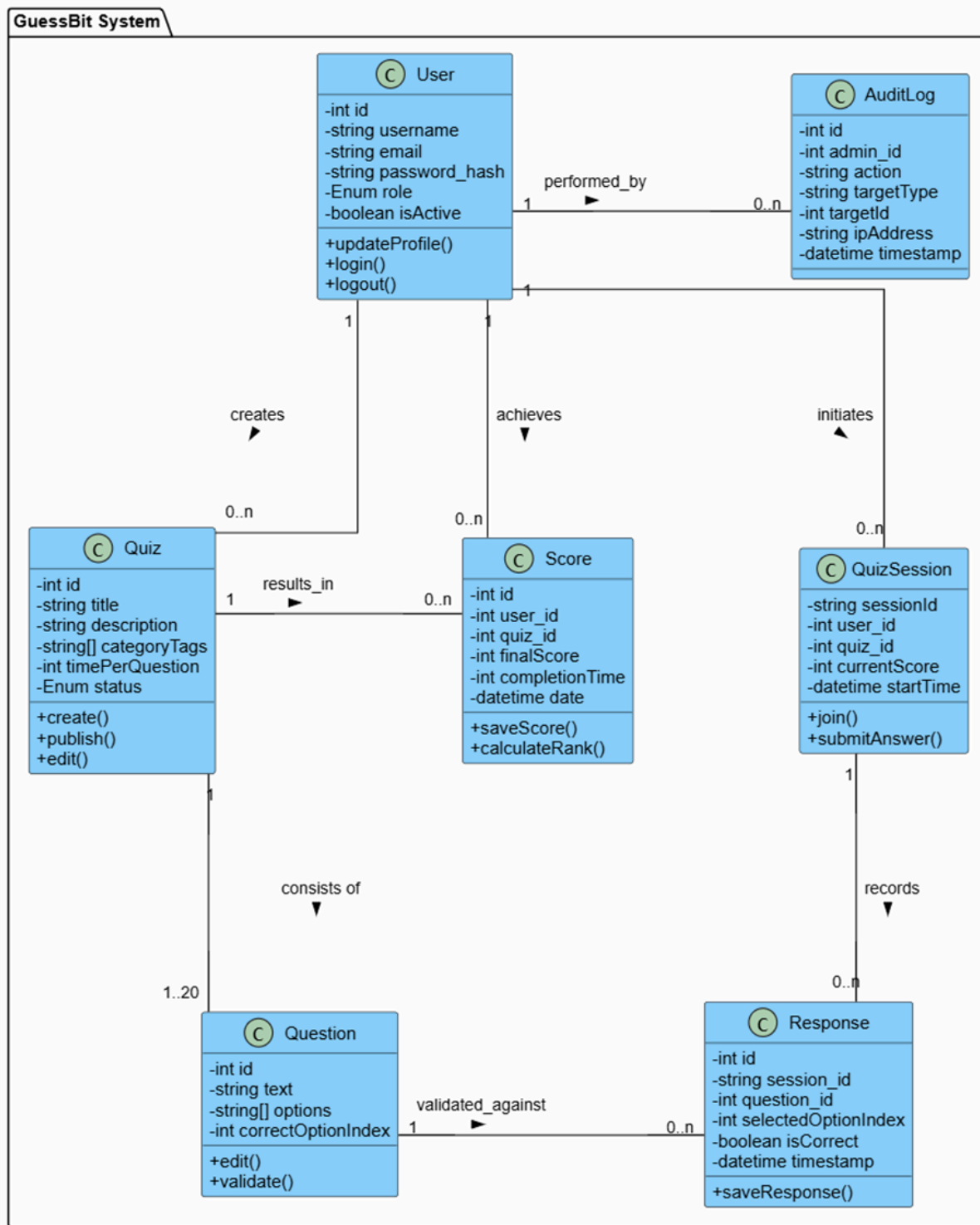
B.3.12 Moderate Content



B.3.13 View System Statistics



B.4 CLASS DIAGRAM



Appendix C: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>